

SIAM EV Platform

API Reference

OCPP Charging Monitoring • Energy Management System • Cargo Space Monitoring

Ruslee Sutthaweeikul

Version 1.0 • July 2, 2026

Contents

1	Introduction	2
1.1	Base URL	2
1.2	Authentication	2
1.3	Common Error Schema	2
1.4	System Architecture Overview	3
2	OCPP Charging Monitoring API	4
2.1	Data Models	4
2.1.1	Station Object	4
2.2	Endpoints	4
2.2.1	GET /ocpp/stations — List all stations	4
2.2.2	GET /ocpp/stations/{id} — Get single station	5
2.2.3	POST /ocpp/stations/{id}/command — Send remote command	5
2.2.4	PUT /ocpp/stations/{id}/schedule — Update charging schedule	6
2.3	WebSocket — Live Telemetry Stream	6
3	Energy Management System (EMS) API	7
3.1	Data Models	7
3.1.1	Facility Object	7
3.1.2	Zone Object (EMS — authoritative sensor record)	7
3.1.3	Energy Reading Object (Time-Series)	8
3.2	Endpoints	8
3.2.1	GET /ems/facilities — List facilities	8
3.2.2	GET /ems/facilities/{id} — Get facility details	9
3.2.3	GET /ems/facilities/{id}/readings — Query time-series data	9
3.2.4	GET /ems/facilities/{id}/analytics — Demand analytics	9
3.2.5	GET /ems/facilities/{id}/zones — List zones with sensor data	10
3.2.6	GET /ems/facilities/{id}/zones/{zoneId} — Single zone detail	10
3.2.7	GET /ems/facilities/{id}/zones/{zoneId}/readings — Zone sensor and energy history	10
3.2.8	PUT /ems/facilities/{id}/zones/{zoneId}/thresholds — Update zone thresholds	11
3.2.9	POST /ems/facilities/{id}/demand-response — Trigger demand response	11
3.2.10	PUT /ems/facilities/{id}/power-limit — Set import limit	11
3.3	WebSocket — Live Energy Feed	12

4	Cargo Space Monitoring API	13
4.1	Data Models	13
4.1.1	Truck Object	13
4.1.2	Route Object	13
4.1.3	Trip Object	14
4.1.4	Station Object	14
4.2	Truck Endpoints	15
4.2.1	GET /cargo/trucks — Live truck map data	15
4.2.2	GET /cargo/trucks/{id} — Get single truck	15
4.2.3	GET /cargo/trucks/{id}/trips — Truck trip history	15
4.3	Route Endpoints	16
4.3.1	GET /cargo/routes — List routes	16
4.3.2	GET /cargo/routes/{id} — Route details	16
4.3.3	GET /cargo/routes/{id}/trips — Trips for a route	16
4.4	Trip Endpoints	17
4.4.1	GET /cargo/trips — Search trips	17
4.4.2	GET /cargo/trips/{id} — Get full trip detail	17
4.5	Planning Analytics Endpoints	17
4.5.1	GET /cargo/analytics/utilization — Load utilization over time	17
4.5.2	GET /cargo/analytics/planning — Fleet planning recommendations	18
4.5.3	GET /cargo/analytics/load-history — Load trend with photos	19
4.6	Station Endpoints	20
4.6.1	GET /cargo/stations — List stations	20
4.6.2	GET /cargo/stations/{id} — Get station	20
4.7	Alert Endpoints	20
4.7.1	GET /cargo/alerts — List active alerts	20
4.7.2	POST /cargo/alerts/{alertId}/acknowledge — Acknowledge alert	20
4.8	WebSocket — Live Cargo and Fleet Feed	21
5	Cross-System Integration	22
5.1	Smart Charging with EMS	22
5.2	Station Energy and Cargo Operations (EMS + Cargo)	22
5.3	Cold-Chain Alert to Fleet Dispatch	23
5.4	Shared Webhook Contract	23
6	API Quick Reference	24

1 Introduction

This document describes the REST and WebSocket APIs exposed by the SIAM EV Unified Energy & Logistics Management Platform. Three sub-systems are covered:

1. **OCPP Charging Monitoring** — real-time telemetry, remote commands, and scheduling for EV charging stations (OCPP 2.0.1).
2. **Energy Management System (EMS)** — facility-level energy analytics, demand-response controls, and renewable-source integration.
3. **Cargo Space Monitoring** — container/cargo-bay occupancy, temperature and humidity telemetry, and access-event logging for logistics hubs.

1.1 Base URL

All endpoints are served under the API Gateway. Replace `{host}` with the deployment address (e.g. `api.siam-ev.example.com`).

```
1 https://{host}/api/v1
```

Listing 1: Base URL pattern

1.2 Authentication

Every request must carry a Bearer token obtained from the Auth Service.

```
1 Authorization: Bearer <access_token>
```

Listing 2: Authorization header

Header	Required	Description
Authorization	Yes	Bearer <JWT> issued by <code>/auth/token</code>
Content-Type	Yes (POST/PUT)	<code>application/json</code>
X-Request-ID	No	Idempotency key; echoed in response headers

1.3 Common Error Schema

```
1 {
2   "success": false,
3   "error": {
4     "code": "STATION_NOT_FOUND",
5     "message": "No station with id 'station-bkk-099' exists.",
6     "timestamp": "2026-07-02T10:15:30Z"
7   }
8 }
```

Listing 3: Error response envelope

HTTP Status	Meaning	Common cause
200	OK	Successful read
201	Created	Resource created
400	Bad Request	Missing or invalid fields
401	Unauthorized	Missing / expired token
403	Forbidden	Insufficient role
404	Not Found	Resource does not exist
409	Conflict	Duplicate resource
500	Internal Error	Upstream / database failure

1.4 System Architecture Overview

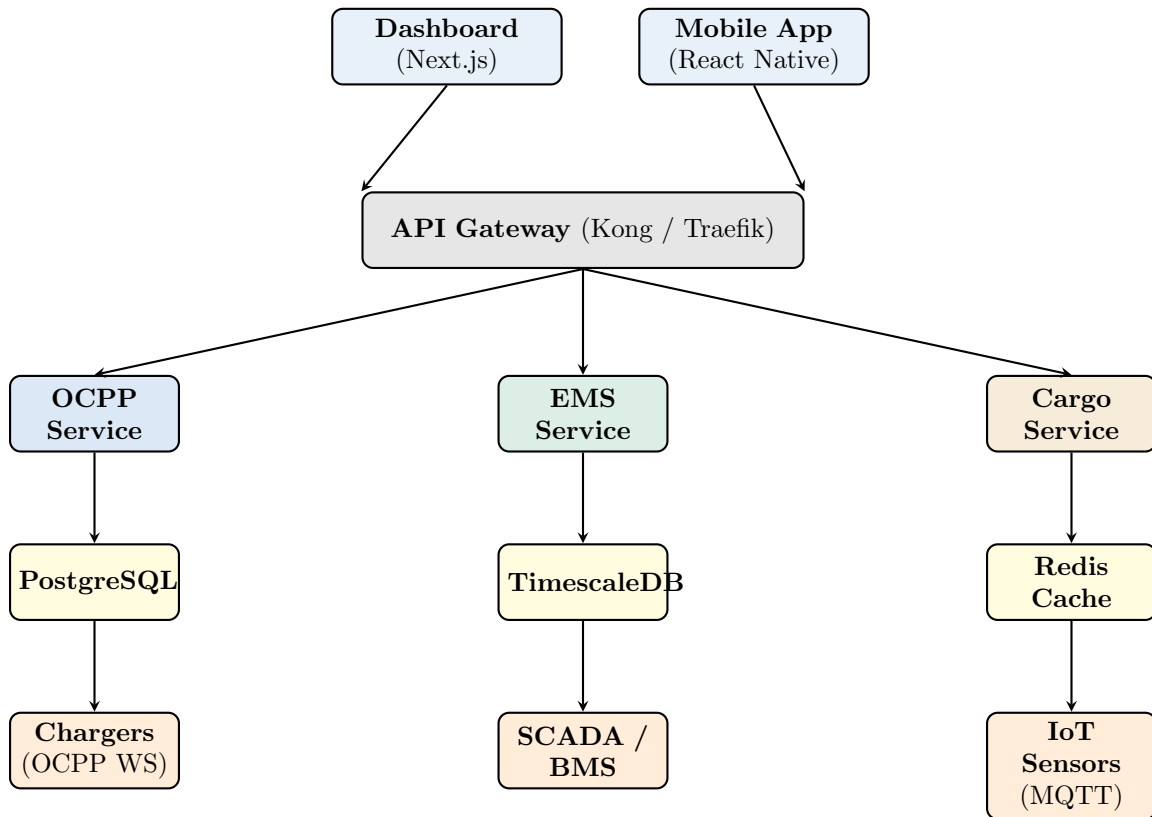


Figure 1: SIAM EV Platform — high-level integration architecture

2 OCPP Charging Monitoring API

Base path: /api/v1/ocpp

This sub-system exposes real-time telemetry from EV charging stations and accepts remote control commands conforming to OCPP 2.0.1. The backend implementation uses Node-RED as the Central System (CS); see `OCPP_NodeRED_Implementation.pdf` for the Node-RED flow details.

2.1 Data Models

2.1.1 Station Object

```

1 {
2   "id":          "station-bkk-001",          // string --- unique station identifier
3   "name":        "Siam Paragon Hub",         // string
4   "lat":         13.7462,                   // number --- latitude
5   "lng":         100.5347,                  // number --- longitude
6   "status":      "Available",               // enum --- see Status Codes below
7   "connectors": [                          // array --- one entry per plug
8     {
9       "connectorId": 1,
10      "type":         "CCS2",
11      "maxPowerKw":   150,
12      "status":      "Available"
13    }
14  ],
15  "currentMeter":  42.15,                   // number --- session energy (kWh)
16  "powerOutput":   68.4,                   // number --- live power draw (kW)
17  "powerLimit":    120,                   // number --- configured cap (kW)
18  "firmwareVersion": "2.1.4",              // string
19  "lastHeartbeat": "2026-07-02T09:58:00Z", // ISO-8601
20  "schedule": {
21    "startTime": "18:00",                  // HH:MM local
22    "endTime":   "08:00",                  // HH:MM local
23    "enabled":   true
24  }
25 }
```

Listing 4: Station object schema

status value	Meaning
Available	Idle, ready to charge
Preparing	EV plugged in, waiting for auth
Charging	Active charging session
SuspendedEV	Car paused session
Finishing	Session ending
Faulted	Hardware fault
Unavailable	Station taken offline

2.2 Endpoints

2.2.1 `GET` /ocpp/stations — List all stations

Returns the current telemetry snapshot for every registered charging station.

Query parameters:

Param	Type	Default	Description
status	string	—	Filter by status enum value
limit	integer	100	Max records returned
offset	integer	0	Pagination offset

Response 200:

```

1 {
2   "success": true,
3   "data": [ /* array of Station objects */ ],
4   "meta": { "total": 12, "limit": 100, "offset": 0 }
5 }

```

Listing 5: GET /ocpp/stations response

2.2.2 GET /ocpp/stations/{id} — Get single station

Path parameter: id — the station identifier string.

Response 200:

```

1 {
2   "success": true,
3   "data": { /* Station object */ }
4 }

```

Listing 6: GET /ocpp/stations/id response

Response 404:

```

1 { "success": false, "error": { "code": "STATION_NOT_FOUND", ... } }

```

2.2.3 POST /ocpp/stations/{id}/command — Send remote command

Dispatches an OCPP remote command to the station.

Request body:

```

1 {
2   "action": "RemoteStopTransaction", // or RemoteStartTransaction | Reset | ...
3   "payload": {
4     "transactionId": 7841 // required for RemoteStopTransaction
5   }
6 }

```

Listing 7: Remote command request

Supported action values:

action	Description
RemoteStartTransaction	Begin a charging session
RemoteStopTransaction	Stop an active session
Reset	Reboot the charge point ("Hard" or "Soft")
ChangeAvailability	Set connector to Available / Unavailable
SetChargingProfile	Push a smart-charging profile (kW limit)

Response 200:

```

1 {
2   "success": true,
3   "data": {
4     "status": "Accepted",
5     "action": "RemoteStopTransaction"
6   }
7 }

```

Listing 8: Command accepted response

2.2.4 **PUT** /ocpp/stations/{id}/schedule — Update charging schedule

Sets the time-window schedule for a station's smart-charging controller.

Request body:

```

1 {
2   "startTime": "22:00",    // HH:MM --- begin off-peak window
3   "endTime":   "06:00",    // HH:MM --- end off-peak window
4   "enabled":   true
5 }
```

Listing 9: Schedule update request

Response 200:

```

1 { "success": true, "data": { "stationId": "station-bkk-001", "schedule": { ... } } }
```

2.3 WebSocket — Live Telemetry Stream

Connect to receive real-time telemetry push events without polling.

```

1 wss://{host}/api/v1/ocpp/ws?token=<access_token>
```

Listing 10: WebSocket connection

Server-sent event (JSON frame):

```

1 {
2   "event": "MeterValues",
3   "stationId": "station-bkk-002",
4   "data": {
5     "connectorId": 1,
6     "transactionId": 7841,
7     "energy_kWh": 58.34,
8     "power_kW": 72.1,
9     "voltage_V": 400.2,
10    "current_A": 180.1,
11    "timestamp": "2026-07-02T10:05:42Z"
12  }
13 }
```

Listing 11: Telemetry push event

Additional event types:

event	Trigger
StatusNotification	Station or connector status change
HeartBeat	Periodic station heartbeat (every 60 s)
StartTransaction	New session started
StopTransaction	Session ended (includes final kWh)
FirmwareStatusNotification	OTA update progress

3 Energy Management System (EMS) API

Base path: /api/v1/ems

The EMS sub-system aggregates energy consumption data from buildings and facilities, provides time-series analytics, and exposes demand-response controls. Data is persisted in TimescaleDB and read from SCADA / BMS systems over Modbus TCP or MQTT.

Relationship to Cargo Stations. Every truck station in the Cargo sub-system is also an EMS *facility* — they are the same physical site viewed from two operational domains. A single shared identifier (`site-bkk-001`) is used as both the `facilityId` in EMS responses and the `stationId` in Cargo responses. The EMS facility object therefore includes the list of cargo-bay zones it contains, and each EMS energy reading can be scoped to an individual cargo bay via the zone-level endpoints (Section 3.2.5).

3.1 Data Models

3.1.1 Facility Object

```

1 {
2   "id":           "site-bkk-001",           // shared key: facilityId in EMS = stationId in
   Cargo
3   "name":         "Thai Post Sorting Hub --- Lad Krabang",
4   "type":         "LogisticsWarehouse",    // Office | Retail | Warehouse | LogisticsWarehouse
5   "location": {
6     "lat": 13.7284,
7     "lng": 100.7498,
8     "address": "999 Moo 1, Lad Krabang, Bangkok 10520"
9   },
10  // --- EMS energy fields ---
11  "ratedCapacityKva": 2000,
12  "currentDemandKw": 312.4,
13  "peakDemandKw": 1540.0,                 // rolling 12-month peak
14  "dailyEnergyKwh": 7820.5,
15  "solarCapacityKw": 500,
16  "batteryCapacityKwh": 1000,
17  "batteryStateOfCharge": 0.78,           // 0.0--1.0
18  "gridImportKw": -45.2,                 // negative = exporting to grid
19  "demandResponseActive": false,
20  "tariffZone": "TOU-EV",
21  // --- Cargo station cross-reference ---
22  "cargoStation": {
23    "stationId": "site-bkk-001",          // same value; emphasises the shared identity
24    "activeTrucks": 3,                   // trucks currently at this station
25    "cargoBayZones": [
26      { "zoneId": "zone-site-001-cold-01", "name": "Cold Room A",
27        "currentKw": 42.1, "occupancyPct": 68 },
28      { "zoneId": "zone-site-001-bay-02", "name": "Dry Bay 2",
29        "currentKw": 8.3, "occupancyPct": 55 },
30      { "zoneId": "zone-site-001-dock-03", "name": "Loading Dock 3",
31        "currentKw": 12.7, "occupancyPct": 0 }
32    ],
33    "ocppStationIds": ["station-bkk-001", "station-bkk-002"] // EV chargers on-site
34  },
35  "updatedAt": "2026-07-02T10:00:00Z"
36 }
```

Listing 12: Facility object schema

3.1.2 Zone Object (EMS — authoritative sensor record)

A *zone* is a named, bounded space inside a facility (cold room, loading dock, dry-storage bay). All environmental sensor data for zones is owned by EMS and read from the facility's BMS or SCADA system over Modbus TCP or MQTT. The Cargo sub-system does **not** maintain its own sensor pipeline for fixed spaces — it reads zone data from these EMS endpoints.


```

1 {
2   "zoneId":      "zone-site-001-cold-01",
3   "facilityId":  "site-bkk-001",
4   "name":       "Cold Room A",
5   "type":       "ColdRoom",           // ColdRoom | LoadingDock | DryBay | StagingArea
6   // --- Energy (from SCADA / smart sub-meter) ---
7   "currentKw":   42.1,
8   "dailyKwh":    892.6,
9   // --- Environment (from BMS sensors) ---
10  "environment": {
11    "temperatureC": 3.8,           // null if no temp sensor
12    "humidityPct":  84.1,
13    "co2Ppm":       405,
14    "lightLux":     0
15  },
16  "setpointC":    4.0,           // BMS setpoint (ColdRoom only)
17  "thresholds": {
18    "tempMinC":     2.0,
19    "tempMaxC":     8.0,
20    "humidityMaxPct": 90.0,
21    "co2MaxPpm":    1000
22  },
23  "alertActive":  false,
24  "source":       "BMS",         // BMS | SCADA | SmartMeter | Estimated
25  "updatedAt":    "2026-07-02T10:02:00Z"
26 }

```

Listing 13: EMS zone object schema

3.1.3 Energy Reading Object (Time-Series)

```

1 {
2   "facilityId":  "site-bkk-001",
3   "timestamp":   "2026-07-02T10:00:00Z",
4   "intervalMin": 15,           // granularity of reading
5   "activeKw":    315.2,
6   "reactiveKvar": 42.1,
7   "apparentKva": 317.9,
8   "powerFactor": 0.991,
9   "phaseA_V":    228.4,
10  "phaseB_V":     229.1,
11  "phaseC_V":     228.8,
12  "source":       "SCADA"       // SCADA | BMS | SmartMeter | Estimated
13 }

```

Listing 14: Facility-level energy reading schema

3.2 Endpoints

3.2.1 `GET` /ems/facilities — List facilities

Query parameters:

Param	Type	Default	Description
type	string	—	Filter by facility type
limit	integer	50	Max records
offset	integer	0	Pagination offset

Response 200:

```

1 {
2   "success": true,
3   "data": [ /* array of Facility objects */ ],

```

```

4  "meta": { "total": 8, "limit": 50, "offset": 0 }
5  }

```

3.2.2 [GET](#) /ems/facilities/{id} — Get facility details

Returns the latest live snapshot for a single facility.

Response 200:

```

1 { "success": true, "data": { /* Facility object */ } }

```

3.2.3 [GET](#) /ems/facilities/{id}/readings — Query time-series data

Returns historical energy readings for a facility over a given time range.

Query parameters:

Param	Type	Required	Description
from	ISO-8601	Yes	Start of range (UTC)
to	ISO-8601	Yes	End of range (UTC)
interval	string	No	Bucket size: 15m 1h 1d (default 15m)
metric	string	No	Specific field to return (e.g. activeKw)

Response 200:

```

1 {
2   "success": true,
3   "data": {
4     "facilityId": "fac-bkk-001",
5     "interval": "1h",
6     "readings": [
7       { "timestamp": "2026-07-02T00:00:00Z", "activeKw": 180.4, "powerFactor": 0.98 },
8       { "timestamp": "2026-07-02T01:00:00Z", "activeKw": 162.1, "powerFactor": 0.97 }
9     ]
10  }
11 }

```

Listing 15: Time-series readings response

3.2.4 [GET](#) /ems/facilities/{id}/analytics — Demand analytics

Returns aggregated KPIs for the facility over a billing period.

Query parameters: period — day — week — month — year (default month)

Response 200:

```

1 {
2   "success": true,
3   "data": {
4     "facilityId": "fac-bkk-001",
5     "period": "month",
6     "totalConsumptionKwh": 234156.2,
7     "peakDemandKw": 1540.0,
8     "peakTimestamp": "2026-06-14T14:30:00Z",
9     "avgLoadFactorPct": 42.8,
10    "solarGenerationKwh": 31200.0,
11    "selfConsumptionPct": 88.4,
12    "gridImportKwh": 202956.2,
13    "gridExportKwh": 3600.0,
14    "estimatedCostThb": 812400.50,
15    "carbonKgCo2e": 91188.0
16  }
17 }

```

Listing 16: Analytics response

3.2.5 [GET](#) /ems/facilities/{id}/zones — List zones with sensor data

Returns all zones within a facility with their current environmental readings (temperature, humidity, CO₂) and energy draw. This is the **authoritative source** for zone sensor data across the platform; the Cargo sub-system reads from this endpoint rather than maintaining a parallel sensor pipeline.

Response 200:

```

1 {
2   "success": true,
3   "data": {
4     "facilityId": "site-bkk-001",
5     "totalCurrentKw": 312.4,
6     "zones": [
7       {
8         "zoneId": "zone-site-001-cold-01",
9         "name": "Cold Room A",
10        "type": "ColdRoom",
11        "currentKw": 42.1,
12        "dailyKwh": 892.6,
13        "environment": { "temperatureC": 3.8, "humidityPct": 84.1, "co2Ppm": 405 },
14        "setpointC": 4.0,
15        "alertActive": false,
16        "activeTruckDocks": 1 // from Cargo -- trucks currently docked here
17      },
18      {
19        "zoneId": "zone-site-001-dock-03",
20        "name": "Loading Dock 3",
21        "type": "LoadingDock",
22        "currentKw": 12.7,
23        "dailyKwh": 84.3,
24        "environment": { "temperatureC": 31.2, "humidityPct": 62.0, "co2Ppm": 510 },
25        "alertActive": false,
26        "activeTruckDocks": 0
27      }
28    ]
29  }
30 }
```

Listing 17: Zone list with sensor and energy data

3.2.6 [GET](#) /ems/facilities/{id}/zones/{zoneId} — Single zone detail

Returns the full EMS Zone object for one zone, including the complete threshold configuration and latest sensor readings.

Response 200:

```

1 { "success": true, "data": { /* EMS Zone object */ } }
```

3.2.7 [GET](#) /ems/facilities/{id}/zones/{zoneId}/readings — Zone sensor and energy history

Returns time-series readings for a single zone, combining energy (kW) and environmental sensors (temperature, humidity, CO₂) in one response. Accepts the same **from/to/interval** parameters as the facility-level readings endpoint.

Response 200:

```

1 {
2   "success": true,
3   "data": {
4     "facilityId": "site-bkk-001",
5     "zoneId": "zone-site-001-cold-01",
6     "interval": "1h",
7     "readings": [
8       { "timestamp": "2026-07-02T08:00:00Z",
9         "activeKw": 40.2, "temperatureC": 4.1, "humidityPct": 83.5, "co2Ppm": 398 },
10      { "timestamp": "2026-07-02T09:00:00Z",
```

```

11     "activeKw": 44.8, "temperatureC": 3.9, "humidityPct": 85.2, "co2Ppm": 412 }
12   ]
13 }
14 }

```

Listing 18: Zone sensor + energy time-series

The `activeKw` spike at 09:00 above correlates with a truck docking and the cold-room doors opening — a pattern the Cargo planning analytics endpoint uses to attribute energy cost to individual routes.

3.2.8 **PUT** /ems/facilities/{id}/zones/{zoneId}/thresholds — Update zone thresholds

Sets the alert thresholds for a zone. Alerts from this endpoint surface in both the EMS WebSocket (`AlertTriggered`) and the Cargo alert endpoints, since the Cargo system subscribes to EMS zone alerts for its stations.

Request body:

```

1 {
2   "tempMinC":      2.0,
3   "tempMaxC":      8.0,
4   "humidityMaxPct": 90.0,
5   "co2MaxPpm":     1000
6 }

```

Response 200:

```

1 { "success": true, "data": { "zoneId": "zone-site-001-cold-01", "thresholds": { ... } } }

```

3.2.9 **POST** /ems/facilities/{id}/demand-response — Trigger demand response

Activates or deactivates a demand-response event for a facility (e.g. shed non-critical loads to avoid peak demand charges).

Request body:

```

1 {
2   "action":          "Activate",           // Activate | Deactivate
3   "targetReductionKw": 200,                // required for Activate
4   "durationMinutes":  30,                  // required for Activate
5   "priority":         "High",              // High | Medium | Low
6   "notifyOccupants": true
7 }

```

Listing 19: Demand-response command

Response 200:

```

1 {
2   "success": true,
3   "data": {
4     "eventId":      "dr-20260702-001",
5     "status":       "Activated",
6     "activatedAt":   "2026-07-02T14:00:00Z",
7     "expiresAt":     "2026-07-02T14:30:00Z",
8     "projectedSavingKw": 195.3
9   }
10 }

```

3.2.10 **PUT** /ems/facilities/{id}/power-limit — Set import limit

Caps the maximum grid import power draw for a facility (used for Smart Charging integration between the EMS and OCPP layers).

Request body:

```

1 {
2   "limitKw":      800,
3   "reason":       "PeakDemandAvoidance", // PeakDemandAvoidance | GridRequest | Manual
4   "expiresAt":    "2026-07-02T16:00:00Z" // null = permanent until changed
5 }

```

Response 200:

```

1 { "success": true, "data": { "facilityId": "fac-bkk-001", "limitKw": 800, "appliedAt": "..."} }

```

3.3 WebSocket — Live Energy Feed

```

1 wss://{host}/api/v1/ems/ws?facilityId=fac-bkk-001&token=<access_token>

```

Server-sent event frame:

```

1 {
2   "event":        "EnergyReading",
3   "facilityId":    "fac-bkk-001",
4   "data": {
5     "activeKw":    318.7,
6     "solarKw":     52.1,
7     "batteryKw":   -10.0, // negative = discharging (exporting to facility)
8     "gridKw":      276.6,
9     "soc":          0.74,
10    "timestamp":    "2026-07-02T10:05:00Z"
11  }
12 }

```

Listing 20: EMS live push event

event	Trigger
EnergyReading	Every 15 s from SCADA/BMS poller
DemandAlert	Demand approaches >90% of contract capacity
SolarForecast	Hourly PV generation forecast update
BatteryStateChange	SoC crosses 20%, 50%, or 80% threshold
DemandResponseEvent	DR event activated or deactivated

4 Cargo Space Monitoring API

Base path: `/api/v1/cargo`

The Cargo Space Monitoring sub-system tracks two things:

- **Trucks** — mobile units tracked by GPS in real time. Each truck carries a load sensor (0–100%) and a camera that photographs the cargo bay automatically when the door closes at departure.
- **Stations** — static named locations (reference points) where trucks arrive and depart. A station has only an identifier, a name, and a geographic position. All environmental and energy data for the physical site is owned by EMS, not by this sub-system.

Cargo photos are stored in S3-compatible object storage and referenced by time-limited pre-signed URLs.

Relationship to EMS. Each cargo *station* is the same physical site as an EMS *facility*; they share the same identifier (e.g. `site-bkk-001`). The Cargo sub-system uses that identifier only as a foreign key — to look up a station’s name and position, and to let callers cross-query EMS for energy and sensor data at the same site. The Cargo API does not replicate or embed EMS sensor data.

4.1 Data Models

4.1.1 Truck Object

```

1 {
2   "id":           "truck-bkk-007",
3   "plateNumber":  "GKh-1234",
4   "name":         "Truck 07",
5   "type":         "StraightTruck", // StraightTruck | Trailer | Refrigerated | Van
6   "capacityCbm":  40,
7   "status":       "InTransit",    // Idle | InTransit | Loading | Unloading | Offline
8   "currentLoadPct": 72,           // 0-100; from load sensor in cargo bay
9   "position": {
10    "lat":         13.8562,
11    "lng":         100.6211,
12    "headingDeg":  245,
13    "speedKph":    68,
14    "timestamp":   "2026-07-02T10:04:55Z"
15  },
16  "currentTripId": "trip-20260702-0041",
17  "driverId":      "usr-drv-019",
18  "lastSeenAt":    "2026-07-02T10:04:55Z"
19 }
```

Listing 21: Truck object schema

status	Meaning
Idle	At a station, engine off, no active trip
InTransit	Moving between stations
Loading	Door open, cargo being loaded
Unloading	Door open, cargo being unloaded
Offline	GPS/telemetry signal lost > 5 min

4.1.2 Route Object

A *route* defines a named sequence of stations a truck is assigned to serve.

```

1 {
2   "id":           "route-bkk-R12",
3   "name":         "Bangkok South Loop",
4   "description":  "Lad Krabang Hub -> Samut Prakan -> Bang Na -> Return",
5   "waypoints": [
```

```

6  { "order": 1, "stationId": "site-bkk-001",
7    "name": "Lad Krabang Hub", "lat": 13.7284, "lng": 100.7498,
8    "dwellMinutes": 30 },
9  { "order": 2, "stationId": "site-bkk-012",
10   "name": "Samut Prakan Depot", "lat": 13.5990, "lng": 100.5964,
11   "dwellMinutes": 20 },
12  { "order": 3, "stationId": "site-bkk-005",
13   "name": "Bang Na Post Office", "lat": 13.6607, "lng": 100.6050,
14   "dwellMinutes": 15 }
15 ],
16 "estimatedDurationMin": 130,
17 "distanceKm": 68.4,
18 "activeOnWorkdays": true,
19 "activeOnWeekends": false
20 }

```

Listing 22: Route object schema

4.1.3 Trip Object

A *trip* is one end-to-end run of a route by a specific truck. A cargo photo is captured automatically on each DoorClose event (i.e. when the door shuts after loading at a station).

```

1  {
2    "id": "trip-20260702-0041",
3    "truckId": "truck-bkk-007",
4    "routeId": "route-bkk-R12",
5    "driverId": "usr-drv-019",
6    "status": "InProgress", // Scheduled | InProgress | Completed | Cancelled
7    "isWorkday": true,
8    "scheduledDepartureAt": "2026-07-02T08:00:00Z",
9    "actualDepartureAt": "2026-07-02T08:07:00Z",
10   "actualArrivalAt": null, // null while InProgress
11   "events": [
12     {
13       "eventId": "evt-tr-0041-001",
14       "type": "DoorClose", // Arrival | Departure | DoorOpen | DoorClose
15       "stationId": "site-bkk-001",
16       "stationName": "Lad Krabang Hub",
17       "timestamp": "2026-07-02T08:05:00Z",
18       "loadPct": 85,
19       "cargoPhoto": {
20         "photoId": "photo-0041-001",
21         "url": "https://storage.siam-ev.example.com/...",
22         "urlExpiresAt": "2026-07-02T11:05:00Z", // pre-signed URL TTL
23         "takenAt": "2026-07-02T08:05:03Z",
24         "fileSizeKb": 342
25       }
26     },
27     {
28       "eventId": "evt-tr-0041-002",
29       "type": "Arrival",
30       "stationId": "site-bkk-012",
31       "stationName": "Samut Prakan Depot",
32       "timestamp": "2026-07-02T09:44:00Z",
33       "loadPct": 61,
34       "cargoPhoto": null
35     }
36   ]
37 }

```

Listing 23: Trip object schema

4.1.4 Station Object

A *station* is a named geographic reference point where trucks pick up and drop off cargo. It is static master data: the Cargo sub-system stores only the fields a dispatcher or map view needs to identify and locate the

site. Environmental and energy data for the same physical site is available via EMS using the same id.

```

1 {
2   "id":      "site-bkk-001",           // shared key: = facilityId in EMS
3   "name":    "Lad Krabang Hub",
4   "address": "999 Moo 1, Lad Krabang, Bangkok 10520",
5   "lat":     13.7284,
6   "lng":     100.7498,
7   "type":    "Hub"                    // Hub | Depot | PostOffice | CustomerSite
8 }

```

Listing 24: Station object schema

4.2 Truck Endpoints

4.2.1 [GET](#) /cargo/trucks — Live truck map data

Returns all trucks with current GPS position and load percentage for the real-time map dashboard.

Query parameters:

Param	Type	Default	Description
status	string	—	Filter by status enum
routeId	string	—	Filter trucks assigned to a route
limit	integer	200	Max records
offset	integer	0	Pagination offset

Response 200:

```

1 {
2   "success": true,
3   "data": [ /* array of Truck objects */ ],
4   "meta": { "total": 18, "inTransit": 11, "idle": 5, "offline": 2 }
5 }

```

Listing 25: Live truck list response

4.2.2 [GET](#) /cargo/trucks/{id} — Get single truck

Returns the full Truck object including the current trip reference.

Response 200:

```

1 { "success": true, "data": { /* Truck object */ } }

```

4.2.3 [GET](#) /cargo/trucks/{id}/trips — Truck trip history

Returns completed and in-progress trips for a specific truck, with optional date and day-type filters.

Query parameters:

Param	Type	Required	Description
from	ISO-8601	No	Filter trips departing after this time
to	ISO-8601	No	Filter trips departing before this time
dayType	string	No	workday weekend all (default all)
routeId	string	No	Limit to a specific route
status	string	No	Completed InProgress Cancelled
limit	integer	No	Max records (default 50)
offset	integer	No	Pagination offset

Response 200:


```

1 {
2   "success": true,
3   "data": [
4     {
5       "id": "trip-20260702-0041",
6       "routeId": "route-bkk-R12",
7       "routeName": "Bangkok South Loop",
8       "status": "InProgress",
9       "isWorkday": true,
10      "actualDepartureAt": "2026-07-02T08:07:00Z",
11      "actualArrivalAt": null,
12      "loadAtDeparturePct": 85,
13      "loadAtArrivalPct": null,
14      "eventCount": 2
15    }
16  ],
17  "meta": { "total": 143, "limit": 50, "offset": 0 }
18 }

```

Listing 26: Truck trip history response

4.3 Route Endpoints

4.3.1 [GET](#) /cargo/routes — List routes

Query parameters: activeOnWorkdays (bool), activeOnWeekends (bool).

Response 200:

```

1 { "success": true, "data": [ /* array of Route objects */ ] }

```

4.3.2 [GET](#) /cargo/routes/{id} — Route details

Returns the full Route object including all waypoints with station coordinates.

Response 200:

```

1 { "success": true, "data": { /* Route object */ } }

```

4.3.3 [GET](#) /cargo/routes/{id}/trips — Trips for a route

Returns all trips that ran (or are running) on a given route, across any truck. Supports the same from/to/dayType filters as the truck-level history endpoint.

Response 200:

```

1 {
2   "success": true,
3   "data": [
4     {
5       "id": "trip-20260702-0041",
6       "truckId": "truck-bkk-007",
7       "truckPlate": "gk-1234",
8       "driverId": "usr-drv-019",
9       "status": "InProgress",
10      "isWorkday": true,
11      "actualDepartureAt": "2026-07-02T08:07:00Z",
12      "loadAtDeparturePct": 85
13    }
14  ],
15  "meta": { "total": 92, "limit": 50, "offset": 0 }
16 }

```

Listing 27: Route trip history response

4.4 Trip Endpoints

4.4.1 `GET` `/cargo/trips` — Search trips

Global trip search across all trucks and routes with rich filtering — the primary endpoint for the history view and the planning dashboard.

Query parameters:

Param	Type	Required	Description
from	ISO-8601	No	Departure time lower bound
to	ISO-8601	No	Departure time upper bound
dayType	string	No	workday weekend all
truckId	string	No	Scope to one truck
routeId	string	No	Scope to one route
stationId	string	No	Trips that visited this station
status	string	No	Trip status filter
minLoadPct	integer	No	Return trips where departure load $\geq$ value
maxLoadPct	integer	No	Return trips where departure load $\leq$ value
limit	integer	No	Max records (default 50, max 500)
offset	integer	No	Pagination offset

Response 200:

```

1 {
2   "success": true,
3   "data": [ /* array of Trip summary objects (no events array) */ ],
4   "meta": {
5     "total":      320,
6     "limit":      50,
7     "offset":     0,
8     "avgLoadPct": 71.4,
9     "workdayCount": 234,
10    "weekendCount": 86
11  }
12 }
```

Listing 28: Trip search response (summary list)

4.4.2 `GET` `/cargo/trips/{id}` — Get full trip detail

Returns the complete Trip object including every event, load reading, and cargo photo URL for that trip.

Response 200:

```

1 { "success": true, "data": { /* full Trip object with events array */ } }
```

Note on cargo photo URLs: Pre-signed URLs embedded in the response expire after 3 hours. Clients should not persist the URL; re-fetch the trip to get a fresh URL if needed.

4.5 Planning Analytics Endpoints

These endpoints aggregate historical trip data into the metrics needed to plan future route assignments (truck count, trip frequency, load efficiency).

4.5.1 `GET` `/cargo/analytics/utilization` — Load utilization over time

Returns average load percentage and trip frequency bucketed by time interval and optionally split by route, truck, or day type.

Query parameters:

Param	Type	Required	Description
from	ISO-8601	Yes	Start of analysis window
to	ISO-8601	Yes	End of analysis window
groupBy	string	No	route truck dayType hour
routeId	string	No	Scope to one route
dayType	string	No	workday weekend all
interval	string	No	Bucket size: 1d 1w 1mo (default 1d)

Response 200:

```

1 {
2   "success": true,
3   "data": {
4     "window": { "from": "2026-06-01T00:00:00Z", "to": "2026-07-01T00:00:00Z" },
5     "interval": "1d",
6     "groupBy": "route",
7     "series": [
8       {
9         "groupKey": "route-bkk-R12",
10        "groupLabel": "Bangkok South Loop",
11        "buckets": [
12          {
13            "date": "2026-06-01",
14            "tripsCompleted": 3,
15            "avgLoadPct": 74.2,
16            "peakLoadPct": 95,
17            "minLoadPct": 42,
18            "totalKm": 205.2
19          }
20        ]
21      }
22    ]
23  }
24 }
```

Listing 29: Utilization analytics response

4.5.2 GET /cargo/analytics/planning — Fleet planning recommendations

Analyses historical patterns and returns data-driven recommendations for route scheduling, truck count, and travel frequency. This is the primary output for the planning workflow.

Query parameters:

Param	Type	Required	Description
from	ISO-8601	Yes	Historical window start
to	ISO-8601	Yes	Historical window end
routeId	string	No	Limit analysis to one route
targetUtil	number	No	Desired avg load % (default 80)

Response 200:

```

1 {
2   "success": true,
3   "data": {
4     "analysisWindow": { "from": "2026-06-01T00:00:00Z", "to": "2026-07-01T00:00:00Z" },
5     "targetUtilPct": 80,
6     "routes": [
7       {
8         "routeId": "route-bkk-R12",
9         "routeName": "Bangkok South Loop",
10        "workdays": {
11          "avgTripsPerDay": 3.1,
```

```

12     "avgLoadPct": 74.2,
13     "peakDemandHour": "08:00",
14     "currentTrucksUsed": 2,
15     "recommendedTrucks": 2,
16     "recommendedFreqPerDay": 3,
17     "utilizationGapPct": 5.8
18   },
19   "weekends": {
20     "avgTripsPerDay": 1.0,
21     "avgLoadPct": 41.3,
22     "peakDemandHour": "10:00",
23     "currentTrucksUsed": 2,
24     "recommendedTrucks": 1,
25     "recommendedFreqPerDay": 1,
26     "utilizationGapPct": 38.7
27   },
28   "insights": [
29     "Weekend demand is 55% lower than workday demand.",
30     "Reallocating 1 truck from weekends could serve 2 additional workday trips.",
31     "Peak load occurs between 07:00 and 09:00 on workdays."
32   ]
33 }
34 ]
35 }
36 }

```

Listing 30: Planning recommendations response

4.5.3 [GET](#) /cargo/analytics/load-history — Load trend with photos

Returns load readings paired with cargo photo thumbnails for a route or truck over a date range. Designed for the history gallery view in the dashboard.

Query parameters:

Param	Type	Required	Description
from	ISO-8601	Yes	Start date
to	ISO-8601	Yes	End date
routeId	string	No	Filter by route
truckId	string	No	Filter by truck
dayType	string	No	workday weekend all
limit	integer	No	Max events returned (default 100)

Response 200:

```

1 {
2   "success": true,
3   "data": [
4     {
5       "tripId": "trip-20260702-0041",
6       "truckId": "truck-bkk-007",
7       "truckPlate": "gk-1234",
8       "routeName": "Bangkok South Loop",
9       "stationName": "Lad Krabang Hub",
10      "departureAt": "2026-07-02T08:07:00Z",
11      "isWorkday": true,
12      "loadPct": 85,
13      "photo": {
14        "photoId": "photo-0041-001",
15        "url": "https://storage.siam-ev.example.com/...",
16        "thumbnailUrl": "https://storage.siam-ev.example.com/.../thumb",
17        "urlExpiresAt": "2026-07-02T11:07:00Z",
18        "takenAt": "2026-07-02T08:05:03Z"
19      }
20    }
21  ]
22 }

```

```

21   ],
22   "meta": { "total": 187, "limit": 100, "offset": 0 }
23 }

```

Listing 31: Load history with photos response

4.6 Station Endpoints

Stations are static master data. These endpoints populate map markers, filter dropdowns, and the route builder. They carry no real-time data; for energy and sensor data at the same physical site use the EMS endpoints with the same id as the facilityId.

4.6.1 `GET` /cargo/stations — List stations

Query parameters: type, limit, offset.

Response 200:

```

1 {
2   "success": true,
3   "data": [ /* array of Station objects */ ],
4   "meta": { "total": 18 }
5 }

```

4.6.2 `GET` /cargo/stations/{id} — Get station

Returns the Station object (id, name, address, lat, lng, type). Cross-query `GET /ems/facilities/{id}` with the same id for energy and sensor data at this site.

4.7 Alert Endpoints

4.7.1 `GET` /cargo/alerts — List active alerts

Query parameters: stationId, truckId, severity (Critical | Warning), limit, offset.

Response 200:

```

1 {
2   "success": true,
3   "data": [
4     {
5       "alertId":      "alrt-20260702-007",
6       "sourceType":   "Zone",           // Zone | Truck
7       "sourceId":     "zone-stn-001-cold-01",
8       "type":         "TemperatureExceeded",
9       "severity":     "Critical",
10      "value":         9.1,
11      "threshold":     8.0,
12      "unit":          "degC",
13      "triggeredAt":   "2026-07-02T10:22:00Z",
14      "acknowledgedAt": null
15    }
16  ]
17 }

```

Listing 32: Active alerts response

4.7.2 `POST` /cargo/alerts/{alertId}/acknowledge — Acknowledge alert

Request body:

```

1 { "acknowledgedBy": "usr-0007", "note": "Compressor maintenance dispatched." }

```

4.8 WebSocket — Live Cargo and Fleet Feed

A single WebSocket connection delivers both fleet (truck) and station (zone) events. Use the **scope** parameter to subscribe only to the events your dashboard panel needs.

```
1 wss://{host}/api/v1/cargo/ws?scope=trucks,zones&token=<access_token>
```

Truck position and load update:

```
1 {
2   "event": "TruckPositionUpdate",
3   "truckId": "truck-bkk-007",
4   "data": {
5     "lat": 13.8562,
6     "lng": 100.6211,
7     "headingDeg": 245,
8     "speedKph": 68,
9     "loadPct": 72,
10    "timestamp": "2026-07-02T10:04:55Z"
11  }
12 }
```

Listing 33: TruckPositionUpdate event

Cargo photo ready event (emitted seconds after door closes at a station):

```
1 {
2   "event": "CargoPhotoReady",
3   "tripId": "trip-20260702-0041",
4   "truckId": "truck-bkk-007",
5   "data": {
6     "eventId": "evt-tr-0041-001",
7     "stationId": "site-bkk-001",
8     "stationName": "Lad Krabang Hub",
9     "loadPct": 85,
10    "photoId": "photo-0041-001",
11    "thumbnailUrl": "https://storage.siam-ev.example.com/.../thumb",
12    "takenAt": "2026-07-02T08:05:03Z"
13  }
14 }
```

Listing 34: CargoPhotoReady event

All WebSocket event types:

event	scope	Trigger
TruckPositionUpdate	trucks	GPS reading every 5 s while InTransit
TruckStatusChange	trucks	Status transitions (e.g. InTransit → Idle)
TripStarted	trucks	New trip begins (truck departs station)
TripCompleted	trucks	Trip ends at final waypoint
CargoPhotoReady	trucks	Photo captured on DoorClose; thumbnail available
EnvironmentUpdate	zones	Zone sensor reading every 30 s
AlertTriggered	zones	Threshold breach on zone or truck sensor
AlertResolved	zones	Reading returns within threshold

5 Cross-System Integration

Several platform workflows span multiple sub-systems. This section documents the key integration points.

5.1 Smart Charging with EMS

When EMS detects that a facility is approaching its demand cap, it can dynamically reduce the `powerLimit` of all EV chargers at that facility:

1. EMS emits a `DemandAlert` WebSocket event (threshold >90%).
2. Cross-Domain Optimizer calculates available headroom.
3. Optimizer calls `PUT /ocpp/stations/{id}/schedule` for each charger to apply a reduced `SetChargingProfile`.
4. EMS calls `PUT /ems/facilities/{id}/power-limit` to register the applied cap for audit.

```

1 // Step 3 --- SetChargingProfile command body
2 {
3   "action": "SetChargingProfile",
4   "payload": {
5     "connectorId": 0,
6     "csChargingProfiles": {
7       "chargingProfileId": 101,
8       "stackLevel": 1,
9       "chargingProfilePurpose": "TxDefaultProfile",
10      "chargingProfileKind": "Relative",
11      "chargingSchedule": {
12        "chargingRateUnit": "W",
13        "chargingSchedulePeriod": [
14          { "startPeriod": 0, "limit": 60000 } // 60 kW cap
15        ]
16      }
17    }
18  }
19 }
```

Listing 35: Smart-charging coordination payload

5.2 Station Energy and Cargo Operations (EMS + Cargo)

Because every cargo station is also an EMS facility (shared `site-bkk-xxx` identifier), three cross-domain workflows run automatically on the platform.

1 — Energy attribution per route. When a trip event (`TripStarted` or `CargoPhotoReady`) is received, the platform tags the EMS zone energy reading for that time window with the `tripId` and `routeId`. The planning analytics endpoint (`GET /cargo/analytics/planning`) includes the resulting `energyCostThbPerTrip` field so operators can see the full cost (fuel + station energy) of each route.

2 — Cold-room demand-response protection. When EMS needs to shed load during a demand-response event, cold-room zones with `activeTruckDocks > 0` (trucks actively loading cold cargo) are excluded from the shed list. The Cross-Domain Optimizer queries both APIs before computing the shed plan:

1. EMS emits `DemandAlert` (facility demand >90% of contract cap).
2. Optimizer calls `GET /ems/facilities/{id}/zones` to get per-zone demand and `activeTruckDocks` counts.
3. Cold rooms with active docks are flagged `protected = true`.
4. Only non-protected, non-critical loads are included in the DR event payload sent to `POST /ems/facilities/{id}/demand-`

```

1 {
2   "action":           "Activate",
3   "targetReductionKw": 200,
4   "durationMinutes":  30,
5   "excludeZones": [
6     "zone-site-001-cold-01"    // protected: truck actively loading cold cargo
7   ],
8   "priority": "High"
9 }

```

Listing 36: DR event with zone exclusions

3 — Loading-dock energy spike correlation. Every time EMS records a zone `activeKw` spike >20% above baseline for a loading-dock zone, the platform queries `GET /cargo/trucks?stationId={siteId}&status=Loading` to correlate the spike with an active truck. The result is stored in the trip event record as `dockEnergyKwh`, giving operations a per-truck energy footprint at the station level.

5.3 Cold-Chain Alert to Fleet Dispatch

When a cargo zone raises a `Critical` temperature alert on a refrigerated truck compartment, the Fleet Service can be notified to reroute the truck to the nearest facility for maintenance:

1. Cargo Monitoring emits `AlertTriggered` over WebSocket.
2. Alert Service creates a notification and calls the Fleet Service REST API.
3. Fleet Service updates the truck's next waypoint.
4. Cargo Monitoring is polled (or receives a push) to confirm the alert is acknowledged once the driver responds.

5.4 Shared Webhook Contract

All three sub-systems support an outbound webhook for integration with external systems (Slack, PagerDuty, ITSM tools). Register a webhook target via:

```

1 POST /api/v1/webhooks
2 Content-Type: application/json
3
4 {
5   "url":           "https://hooks.example.com/siam-ev",
6   "secret":        "sha256-hmac-secret",
7   "events":        ["cargo.AlertTriggered", "ems.DemandAlert", "ocpp.StopTransaction"]
8 }

```

Listing 37: Register outbound webhook

Webhook payloads are signed with HMAC-SHA256 in the `X-SIAM-Signature` header for verification.

6 API Quick Reference

Endpoint	Method	Description
<i>OCPP Charging Monitoring</i>		
/ocpp/stations	GET	List all stations with telemetry
/ocpp/stations/{id}	GET	Get single station
/ocpp/stations/{id}/command	POST	Remote command (start/stop/reset)
/ocpp/stations/{id}/schedule	PUT	Update charging schedule
/ocpp/ws	WS	Live telemetry stream
<i>Energy Management System</i>		
/ems/facilities	GET	List facilities
/ems/facilities/{id}	GET	Facility live snapshot
/ems/facilities/{id}/readings	GET	Time-series energy readings
/ems/facilities/{id}/analytics	GET	Demand KPIs for billing period
/ems/facilities/{id}/zones	GET	All zones: sensor + energy data (authoritative)
/ems/facilities/{id}/zones/{zoneId}	GET	Single zone detail
/ems/facilities/{id}/zones/{zoneId}/readings	GET	Zone sensor + energy time-series
/ems/facilities/{id}/zones/{zoneId}/thresholds	PUT	Update zone alert thresholds
/ems/facilities/{id}/demand-response	POST	Activate / deactivate DR event
/ems/facilities/{id}/power-limit	PUT	Cap grid import power
/ems/ws	WS	Live energy feed
<i>Cargo Space Monitoring — Fleet</i>		
/cargo/trucks	GET	Live truck positions and loads (map)
/cargo/trucks/{id}	GET	Single truck detail
/cargo/trucks/{id}/trips	GET	Trip history for a truck
/cargo/routes	GET	List routes
/cargo/routes/{id}	GET	Route detail with waypoints
/cargo/routes/{id}/trips	GET	All trips on a route
/cargo/trips	GET	Global trip search with date/dayType filter
/cargo/trips/{id}	GET	Full trip detail with events and photos
<i>Cargo Space Monitoring — Analytics</i>		
/cargo/analytics/utilization	GET	Load % and trip frequency over time
/cargo/analytics/planning	GET	Fleet sizing and frequency recommendations
/cargo/analytics/load-history	GET	Load trend with cargo photo gallery
<i>Cargo Space Monitoring — Stations (static reference data)</i>		
/cargo/stations	GET	List stations (name, position, type)
/cargo/stations/{id}	GET	Single station; use same id with EMS for sensor data
/cargo/alerts	GET	Active alerts (trucks)
/cargo/alerts/{alertId}/acknowledge	POST	Acknowledge an alert
/cargo/ws	WS	Live truck event feed
<i>Shared</i>		
/webhooks	POST	Register outbound webhook